

Writing local tools

LESSON 17 · MODULE 4 — TOOLS & WORKFLOWS

What you'll learn

- How to give an agent **more than one tool** and let *it* decide which to call
- How Python **type hints become the tool's schema** — `int` vs `float`, and defaults that make an argument optional
- How an agent **chains several tool calls in a single turn** to answer one question that needs all of them
- When to reach past a plain function for the explicit **Tool** class (custom schema, enums, a custom name)

Key concepts

One tool was the warm-up; the real pattern is many tools, agent's choice. Last lesson you handed the agent a single function. Here you register two — an inventory lookup and a price calculator — and ask one question that needs both. You never script the order. The agent reads each docstring, decides which tools apply, and calls them. You write plain Python; the agent does the orchestration.

****Type hints are the schema — and they carry real meaning.**** Look at `calculate_quote(quantity: int, unit_price: float, discount_pct: float = 0.0)`. The `int` tells the model to pass a whole number, the `float` a decimal. That's not decoration — it's the contract the model fills in from the user's words. Get the hint right and the model passes the right shape.

A default turns an argument optional. `discount_pct` defaults to `0.0`, so the model only fills it in when the user actually mentions a discount. No default means the model treats the argument as required. That one line is the whole "is this optional?" story — you express it the normal Python way and the SDK reads your signature.

The agent fans out across tools in one turn. Register both with two `add_tool` calls, then ask a question that needs a stock check *and* a quote. In a single `run`, the agent calls `check_inventory`, calls `calculate_quote`, and merges the two results into one answer. Hand it as many tools as you want — it picks the right ones per request and can fire several at once.

Plain function first; **Tool class only when you outgrow it.** Most of the time a plain function with a good docstring and type hints is all you need. When the auto-inferred schema isn't enough — you need enums, a custom tool name, or a hand-written schema — there's a `Tool` class for that. Reach for it as the exception, not the default.

Code shown in this lesson

PYTHON

TAKEAWAY

One tool was the warm-up; the real pattern is many tools, agent's choice.

TAKEAWAY

A default turns an argument optional.

TAKEAWAY

The agent fans out across tools in one turn.

```
# Same notebook setup as the rest of the series – the [jupyter] extra pulls in
# nest_asyncio so tool calls work inside a notebook event loop.
# pip install "lyzr-adk[jupyter]"
from lyzr import Studio

studio = Studio(api_key="...")
```

PYTHON

```
# Tool 1 – inventory lookup, same shape as the last lesson.
def check_inventory(sku: str) -> str:
    """Look up live inventory status for a product SKU."""
    inventory = {"GH-2400": "in stock, 47 units, DC East"}
    return inventory.get(sku, "SKU not found")

# Tool 2 – a price calculator. The signature defines the schema:
#   quantity is a whole number, unit_price a decimal, discount_pct is optional.
def calculate_quote(quantity: int, unit_price: float, discount_pct: float = 0.0) -> str:
    """Calculate a price quote for a number of units, with an optional discount percent."""
    total = quantity * unit_price * (1 - discount_pct / 100)
    return f"{quantity} units @ ${unit_price:.2f} = ${total:.2f} ({discount_pct}% off)"
```

PYTHON

```
# Hand the agent both tools – two add_tool calls.
agent = studio.create_agent(
    name="sales", provider="gpt-4o",
    role="sales assistant",
    goal="answer stock and pricing questions",
)
agent.add_tool(check_inventory)
agent.add_tool(calculate_quote)

# One question that needs both. The agent runs the stock check and the quote,
# then merges them into a single answer – all in one turn.
agent.run("Is GH-2400 in stock, and what's the price for 100 units at $12.50?")
```

TRY THIS

- 1 Ask a question that needs only *one* of the two tools (just a stock check). Confirm the agent calls that tool alone and leaves the other untouched — proof it's choosing, not running everything.
- 2 Mention a discount in your prompt ("...with a 10% discount") and check that `discount_pct` gets filled in. Then drop the discount and confirm it falls back to the `0.0` default.
- 3 Add a third tool — say `get_lead_time(sku: str) -> str` — and ask a question that needs all three. Watch the agent chain the calls and combine the results in one turn.